The propositions package

Cian Dorr (with help from Claude)
ciandorr@gmail.com

Version 1.0: 2026/02/13

Abstract

The propositions package provides a key-value driven system for labelling propositions, theses, and premises in academic papers. Items may be given names like '(P)' or 'Physicalism', or auto-numbered using different counters; all carry robust cross-references with configurable formatting. The package integrates with amsmath, hyperref, and cleveref.

Contents

1 Introduction **2**

2 History **2**

3 Basic usage **2**

4 Advanced usage **3**

5 Basic environments and commands **5**

6 Keys **5**
6.1 Item-level keys 5
6.2 Keys for prop environments 8
6.3 Keys for prop and inlineprop environments 10
6.4 Global keys 11

7 Defining and modifying styles **12**

8 Built-in styles **14**

9 Cross-referencing commands **20**
9.1 How cross-referencing works: \propapply 22

10 Compatibility **23**

11 Known issues **23**

1 Introduction

In some academic disciplines (such as philosophy), it is common to have displayed propositions (examples, theses, premises,...) with various kinds of labels. A thesis might be referred to as '(P)' or 'Physicalism'; the premises of an argument might be numbered as 'P1', 'P2', 'P3', ...; or examples might be numbered consecutively over the course of a whole article. Standard LaTeX environments like `enumerate` can handle some of these cases, but cross-referencing is awkward: `\ref` produces a bare number or letter, and the author must manually add parentheses or other formatting at every point of reference. The standard `description` environment does not allow cross-referencing at all.

The propositions package solves this by attaching formatting information to each label. A short item like `\item[P]` (inside `prop`) is displayed as "(P)" and `\ref` automatically produces "(P)" as well—complete with parentheses, hyperlinks, and cleveref support. The full key-value interface supports named items, numbered items, custom counters, glosses, shorthands, and per-item format overrides.

The `\ptag` command (which requires amsmath) extends this to displayed math environments: an equation can be tagged with a proposition label instead of (or using) its equation number.

2 History

I wrote the ancestor to this package in the 90s while finishing my Ph.D. thesis, but never documented it or shared it with the world. This new version is a thorough re-implementation in LaTeX3, written in 2026 with extensive help from Claude Code. I hope others will find it as useful as I have.

3 Basic usage

Load the package with `\usepackage{propositions}` or `\usepackage[`⟨*options*⟩`]{propositions}` (see subsection 6.4 below for valid package options).

The `prop` environment generates a list of propositions, each introduced by `\item`. `\item` with an optional argument gives a `description`-like label:

```
\begin{prop}
  \item[Physicalism] Everything is physical. \label{phys}
  \item[Idealism] Everything is mental. \label{ideal}
\end{prop}
```

Physicalism Everything is physical.

Idealism Everything is mental.

Unlike the standard `description` environment, one can refer back to these propositions using the standard `\ref` command (or with new cross-referencing commands described in section 9):

```
\ref{phys} is more plausible than \ref{ideal}.
```

Physicalism is more plausible than **Idealism**.

With no optional argument, `\item` will by default generate numbered items similar to `enumerate`, but with numbering that persists across the document:

```
\begin{prop}
  \item Every atom is physical. \label{atoms}
\end{prop}
\ref{phys} follows from the conjunction of \ref{atoms} and
\begin{prop}
  \item Everything is an atom. \label{atomism}
\end{prop}
```

(1) Every atom is physical.

Physicalism follows from the conjunction of (1) and

(2) Everything is an atom.

As with `enumerate`, the counter and formatting depend on the nesting level:

```
\begin{prop}
  \item \label{dual}
  \begin{prop}
    \item Some things are physical. \label{some}
    \item Some things are not physical. \label{notall}
  \end{prop}
\end{prop}
Without \ref{some}, \ref{dual} would be consistent
with \ref{ideal}.
```

(3) a. Some things are physical.

b. Some things are not physical.

Without (3a), (3) would be consistent with **Idealism**.

4 Advanced usage

The format of the proposition labels, and of subsequent references, are both configurable using a key=value syntax (see section 6 for the possible keys):

```
\begin{prop}
  \item[No Overlap,
        align=flush,
        display format=\textsc{#1},
        ref format=\textit{#1}]
    Nothing mental is physical. \label{incomp}
\end{prop}
Is \ref{incomp} consistent with \ref{phys}?
```

No Overlap Nothing mental is physical.

Is *No Overlap* consistent with **Physicalism**?

The prop environment can also take an optional argument with a list of keys:

```
\begin{prop}[leftmargin=5em, display format=[#1]]
  \item Every mental thing is physical.
\end{prop}
```

[4] Every mental thing is physical.

Preset styles can be declared and used in place of a set of key-value pairs:

```
\begin{prop}
  \item[Nihilism, style=thesis] There is nothing. \label{nihilism}
\end{prop}
Does \ref{nihilism} imply \ref{phys}, \ref{dual}, or both?  Discuss.
```

Nihilism There is nothing.

Does Nihilism imply **Physicalism**, (3), or both? Discuss.

Shortcut commands—e.g., `\titem[`⟨*keys*⟩`]` for `\item[style=thesis,` ⟨*keys*⟩`]`—can also be defined. The package loads with a range of predefined styles (listed in section 8).

When the package is loaded with `\usepackage[equations]{propositions}`, a first-level `\item` within `prop` will use the counter as equations. (This looks better with the `leqno` option to `\documentclass`.)

```
\begin{equation}
  \exists x (\text{Mental}(x) \wedge \text{Physical}(x))
\end{equation}
\begin{prop}
  \item There is overlap between the mental and the physical.
\end{prop}
```

(5) $$\exists x(\text{Mental}(x) \wedge \text{Physical}(x))$$

(6) There is overlap between the mental and the physical.

The `\ptag` command (requires `amsmath`) is a replacement for the standard `\tag` command that behaves just like an `\item` in a `prop` environment.

```
\begin{equation}
  \ptag[Monism] \label{mon}
  \exists x \forall y(y = x)
\end{equation}
Is \ref{mon} compatible with \ref{dual}?
```

Monism $$\exists x \forall y(y = x)$$

Is **Monism** compatible with (3)?

5 Basic environments and commands

`\begin{prop}[`⟨*keys*⟩`]`
⟨*environment content*⟩
`\end{prop}`

Creates a displayed list of propositions. It is a standard LaTeX list, so by default its formatting will depend on the standard length parameters like `\itemsep` and `\topsep`, although these can be overridden by setting keys.
Within `prop`, `\item` creates the propositions (see below).
The optional ⟨*keys*⟩ argument can contain a list of keys, which will affect only the given environment (not any other `prop` environments that may be nested within it).

`\begin{inlineprop}[`⟨*keys*⟩`]`
⟨*environment content*⟩
`\end{inlineprop}`

Like `prop`, but does not create a list. Allows `\item` to be used outside list environments, e.g. for generating numbers at the beginning of paragraphs. Steps the `prop` counter and increments the nesting level. Accepts the same optional ⟨*keys*⟩ as `prop`.

Within `prop` and `inlineprop`, `\item` is redefined to act as the proposition-item command. Its optional argument is a comma-separated list of ⟨*key*⟩=⟨*value*⟩ pairs (see section 6). A bare string without = is treated as a proposition name.

When used without an optional argument (or without setting `name`, `counter`, or `style`), the style is determined by the `nameless style` key (section 6). By default this is `numbered`, which dispatches by nesting depth to `levelone`, `leveltwo`, ... via `\proplevelchoice`.

`\ptag[`⟨*keys*⟩`]`

(Available only when `amsmath` is loaded.) Works inside displayed math environments like `equation` and `align`. Accepts the same keys as `\item` within `prop`, except that `align` has no effect (since positioning is controlled by the tag placement system).
Any display math environment (`equation`, `align`, etc.) used inside `prop` or `inlineprop` automatically increments the nesting level for its duration, so that `\ptag` inside such an environment behaves as if it were one level deeper.

`\propoptions{`⟨*keys*⟩`}`

Sets default keys (see section 6), which take effect for all subsequent uses of `prop`, `inlineprop`, `\item`, and `\ptag` (within the current scope).

Further commands are described below, especially in section 7 (defining styles) and section 9 (cross-referencing).

6 Keys

6.1 Item-level keys

The keys listed in this subsection can be used in the following places:

- In the optional argument of `\item` or `\ptag`: the key applies to that item only.
- In the optional argument of `prop` or `inlineprop`: the key applies to every top-level `\item` within that environment, unless overridden by the `\item`'s own optional argument. (If another `prop` or `inlineprop` environment is used inside this one, the key will not apply to its `\item`s.)
- In the argument of `\propoptions` → P. 5: the key will apply to every subsequent `\item` or `\ptag` (in the current group), unless overridden by that `\item`'s optional argument, or that of its parent `prop` or `inlineprop` environment.
- In the optional argument of `\usepackage{propositions}`: equivalent to `\propoptions`.
- With `\SetPropStyle` or `\DeclareNumberedStyle`, to add the key to a given style.

`style`=⟨*style*⟩ (no default)

A prop style, equivalent to a preset collection of keys: see section 7 for details.

`align`=⟨*value*⟩ (no default)

How the label should be positioned. Possible values: `left` (standard left-aligned label, offset controlled by `\labelwidth` and `\labelsep`), `right` (right-aligned, like `enumerate`), `center` (centered within the label box), `flush` (aligned with left margin of item text), `nextline` (label on its own line), and `flush-nextline`. Has no effect inside `inlineprop` or `\ptag`.

```
\begin{prop}
  \item[A, align=left] Left aligned label.
  \item[A, align=center] Center aligned label.
  \item[A, align=right] Right aligned label.
  \item[Long label, align=center] As in standard \LaTeX\ lists,
      longer labels expand to fill the label box and then push
      the following text along to make room for themsselves.
  \item[A, align=flush] Flush aligned label.
  \item[But sometimes a long label deserves its own line,
      align=nextline] Nextline aligned label.
  \item[Another rather long label,
      align=flush-nextline] Flush-nextline aligned label.
\end{prop}
```

A Left aligned label.

A Center aligned label.

A Right aligned label.

Long label As in standard LaTeX lists, longer labels expand to fill the label box and then push the following text along to make room for themsselves.

A Flush aligned label.

But sometimes a long label deserves its own line
Nextline aligned label.

Another rather long label

Flush-nextline aligned label.

`name=`⟨*text*⟩ (no default)

The proposition's name. A bare string (without `=`) is equivalent to `name=`⟨*text*⟩. (To be precise: if the optional argument contains no `=` at all, the whole of it is the name; otherwise, it is read as a key list, and any entry of it without an `=` is the name, so a bare name can be combined with other keys, as in `\item[No Overlap, align=flush]`. In that case a name containing a comma must be braced—`\item[{Alpha, Beta}, align=flush]`—and a name containing an `=` needs the explicit `name={`⟨*text*⟩`}` form.)

`counter=`⟨*name*⟩ (no default)

Counter to use. The counter is automatically stepped, and the item's `name` is set to `\the`⟨*name*⟩ (though this can be overridden by `counter format` or `name`).

`counter format=`⟨*template*⟩ (no default)

How to display the counter value. Use `#1` for the counter name, e.g. `counter format=\roman{#1}`. When both this key and `counter` are set, this is used instead of `\the`⟨*counter*⟩ for generating the item's `name`.

`format=`⟨*template*⟩ (no default)

Formatting applied to the `name`: use `#1` for the argument, e.g. `format=\textbf{(#1)}`. Shorthand for setting both `display format` and `ref format`.

`display format=`⟨*template*⟩ (no default)

Format for displaying the name in the proposition's label. Does not affect cross-references.

`ref format=`⟨*template*⟩ (no default)

Format for subsequent cross-references to this proposition. Does not affect the display.

`shorthand=`⟨*text*⟩ (no default)

An abbreviation displayed after the name. If present, the shorthand becomes the reference text: `\ref` produces the shorthand (formatted with `ref format`) rather than the full name.

`shorthand format=`⟨*template*⟩ (initially `~[#1]`)

Format for displaying the shorthand in the label.

`gloss=`⟨*text*⟩ (no default)

A parenthetical gloss displayed after the name. Does not affect cross-references.

`gloss format=`⟨*template*⟩ (initially `~(#1)`)

Format for displaying the gloss in the label.

`label format=`⟨*template*⟩ (no default)

Format applied to the *entire* assembled label, i.e. the name (after `display format`), shorthand, and gloss together. Use `#1` for the whole assembled text. For example, `label format=#1:` appends a colon after the complete label.

`ref`=⟨*text*⟩ (no default)

Explicitly set the reference text, overriding what would be derived from `name`, `counter`, or `shorthand`.

`label`=⟨*label*⟩ (no default)

Equivalent to a trailing `\label{`⟨*label*⟩`}`.

`reset`=⟨*boolean*⟩ (initially `true`)

When `true` (the default), processing an `\item` resets the sub-level counter one nesting depth below the current level (e.g. `numpropii` at level 1). `reset=false` suppresses this behaviour, so that sub-item numbering continues from where it left off across consecutive parent items.

`crefname`=⟨*type*⟩ (no default)

When cleveref is loaded, assigns an arbitrary reference type to this proposition. For example, `crefname=lemma` causes `\cref` to use the names defined by `\crefname{lemma}{...}{...}` instead of the default (private) `prop` type. The ⟨*type*⟩ must be known to cleveref; new types can be declared with `\crefname`.

Four of the list-dimension keys documented in the next section—`labelwidth`, `labelsep`, `itemindent` and `labelindent`—may also be given to an individual `\item`, where they reposition the label of that item alone, overriding the value in force for the rest of the list. (The other dimension keys have no effect at item level.)

6.2 Keys for `prop` environments

The keys in this section can be used in the following places:

- In the optional argument of `prop`: the key applies to that environment only, not to any other `prop` or `inlineprop` environments that may be nested within it.
- In the argument of `\propoptions` → P. 5: the key will apply to every subsequent `prop` environment (in the current group), unless overridden by that environment's optional argument.
- In the optional argument of `\usepackage{propositions}`: equivalent to `\propoptions`.
- With `\SetPropStyle` or `\DeclareNumberedStyle`, to add the key to a given style.

The keys can also be set in the optional argument of `\item`, `\ptag`, or `inlineprop`, or as part of a style used in one of those contexts, but will have no effect there, except for `labelwidth`, `labelsep`, `itemindent` and `labelindent` which will override the given dimensions for an individual `\item`.

`topsep`=⟨*length / length list*⟩
`partopsep`=⟨*length / length list*⟩
`itemsep`=⟨*length / length list*⟩

`parsep`=⟨*length* / *length list*⟩
`leftmargin`=⟨*length* / *length list*⟩
`rightmargin`=⟨*length* / *length list*⟩
`labelwidth`=⟨*length* / *length list*⟩
`labelsep`=⟨*length* / *length list*⟩
`itemindent`=⟨*length* / *length list*⟩
`listparindent`=⟨*length* / *length list*⟩

Override the standard LaTeX list dimensions. Accept the same values as `\setlength`, including rubber lengths (e.g. `itemsep=4pt plus 2pt`).
Each also accepts the value `*`, which means *as if the key had not been set*: the dimension takes the document class's default, or—where `labelindent` makes the five label-area dimensions over-determined—is left free for the package to derive from the others. This is the same convention enumitem uses, and it is chiefly useful for confining an override to some levels of nesting, since a value is resolved afresh at each level:

```
\propoptions{leftmargin = \proplevelchoice{2.5em, 0em, *}}
```

gives level 1 2.5em, level 2 0em, and every deeper level the class default. Without the `*` the last entry would apply at every level below the second, since `\proplevelchoice` → P. 13 clamps rather than running out.
When used with a `prop` environment, the values of these keys can contain the special `\propformlabel` macro: see below.

`\propformlabel`

Expands to the formatted label of the (approximately) `items`-th item in this environment, computed at `\begin{prop}` *before* the list dimensions are applied, so it can be used directly in dimension expressions. Updated after each `\item` to reflect the most recently output label. Typical use:

```
\begin{prop}[items=20,
  leftmargin=\widthof{\propformlabel}+0.5em]
```

sizes the left margin to accommodate labels up to the 20th item.

`\propwidestlabel`

`\propformlabel` with every digit replaced by the digit that makes the label widest (an `8` in the usual case, where all digits have the same width). Using it in place of `\propformlabel` in a dimension expression makes the dimension depend on how many digits the label has rather than on which digits they are, so that a run of numbered environments keeps a steady margin instead of shifting at every change of number: `(9)` and `(10)` differ in width by 5pt in the default font, where `(8)` and `(88)` differ only by as much as an extra digit requires. (The substitution is used for measurement only; the labels themselves are untouched.) This is what the `fitmargin` style measures.

`\propwidest{`⟨*label*⟩`}`

The filter that `\propwidestlabel` applies: expands ⟨*label*⟩ and replaces all of its digits by whichever single digit makes it widest, leaving formatting, letters and spaces alone. Use it on a label of your own, e.g. `\widthof{\propwidest{(\ref{lastitem})}}`.

The widest digit is found by measurement rather than assumed, since it depends on the font: with tabular figures every digit has the same width and the choice is immaterial, but with proportional (usually oldstyle) figures it is not, and `8` is then often not the widest—in EB Garamond's oldstyle figures it is only the sixth widest, narrower than `0` by half a point per digit, enough for a margin reserved for `(88)` to let `(00)` overflow it. The measurement costs eleven boxes, incurred only where `\propwidest` is used.

`items=`⟨*n*⟩ (initially `1`)

Indicates that approximately ⟨*n*⟩ items are expected in this environment. The only effect of this key is in conjunction with `\propformlabel` → P. 9, to compute a preview label wide enough for the ⟨*n*⟩th expected item, so that dimension expressions such as `leftmargin=\widthof{\propformlabel}+0.5em` reserve enough space for the widest label. Has no effect on actual item processing or counter stepping.

`labelindent=`⟨*length / length list*⟩ (no default)

Positions the left edge of the label box at ⟨*length*⟩ from the enclosing margin, adjusting `\labelwidth`, `\leftmargin`, `\labelsep`, or `\itemindent` as needed. (Since the value of any one of these dimensions can be derived from those of the other four, it never makes sense to set all five.)

`tightspacing` (no value)

Sets all vertical spacing to the compact defaults that the standard document classes use for level-three lists (`topsep` and `itemsep` to `2pt` with stretch/shrink, `parsep` to `0pt`, `partopsep` to `1pt`).

`nosep` (no value)

Sets `\topsep`, `\itemsep`, and `\parsep` all to zero.

`continue=`⟨*boolean*⟩ (initially `true`)

When `true`, a `prop` environment that immediately follows another `prop` (with no intervening paragraph text) replaces the normal `\topsep`-based inter-list space with `\itemsep` + `\parsep`, giving the visual appearance of a single continuous list.

6.3 Keys for `prop` and `inlineprop` environments

The keys in this section have effect in both the `prop` and `inlineprop` environments. They can also be used with `\propoptions`, `\usepackage`, and `\SetPropStyle`, just like the keys in the previous subsection.

`named style=`⟨*style*⟩ (initially `proposition`)

The style when an `\item` or `\ptag` has a value for `name` but no explicit `style`.

`named ptag style=`⟨*style*⟩ (initially empty)

If set, overrides `named style` for `\ptag` items only.

`nameless style=`⟨*style*⟩ (initially `numbered`)

The style used when `\item` or `\ptag` has no value for `name` and no explicit `style`. A `counter` does not affect the choice: it says how the item is numbered, and this is the style meant to look right around a number.

`nameless ptag style=`⟨*style*⟩ (initially empty)

If set, overrides `nameless style` for `\ptag` items only.

The choice between them turns on `name` alone, read from the whole key list—`\propoptions`→P. 5, then the environment argument, then the item's own argument. An item counts as named if a name reaches it from any of those, so `\begin{prop}[name=foo]\item bar` gives the same result as `\begin{prop}\item[name=foo] bar`. An explicit `style` overrides the choice wherever it is set.

`wrapper begin=`⟨*code*⟩
`wrapper end=`⟨*code*⟩

Insert arbitrary ⟨*code*⟩ immediately before the beginning and after the end of the environment, so that the whole list can (for example) be wrapped in another environment.

Any value that contains a comma (such as a comma-separated list of options to an environment) must be enclosed in braces, so that the comma is not read as a key separator:

```
\begin{prop}[
  wrapper begin = {\begin{tcolorbox}[
      colframe = red, colback = white]},
  wrapper end   = {\end{tcolorbox}}]
  \item[Red] A proposition in a red frame.
  \item And a second one, in the same frame.
\end{prop}
```

Red A proposition in a red frame.
(7) And a second one, in the same frame.

The wrapping environment must be one that can be split into separate begin and end code—that is, it must not read its body as a macro argument via `\collect@body` or similar.

The following command is provided for use with `wrapper begin`:

`\propoperativeleftmargin`

A read-only length giving the `leftmargin` a `prop` environment (with no optional argument) would use, given the current defaults set by `\propoptions` and the current nesting level. It is recomputed at every `\begin{prop}`, so it is valid inside a wrapper (before the environment's own `leftmargin` has taken effect). For example, the built-in `framed` style uses this command in the value of `wrapper begin` to make the margins of framed environments match those of regular environments.

6.4 Global keys

The following key can only be set in the optional argument of `\usepackage`, or in the preamble with `\propoptions`.

`equations` (no value, **global only**)

Installs hooks so that the LaTeX and `amstex` displayed equation environments (such as `equation` and `align`) use the same formatting as the special `equation` prop style. The `equation` style's `display format` and `ref format` keys are used for generating equation numbers and cross-references to them. Redefining the `equation` style, e.g. with

```
\SetPropStyle{equation}{format = [#1]}
```

will automatically update the hooks.

The `equations` key also sets `nameless style=eqnum`, so that top-level `\item`s with no name will also use the `equation` style, while lower-level `\item`s will use other kinds of numbering: for details, see section 8 below.

7 Defining and modifying styles

Styles, equivalent to bundles of key-value settings, can be defined, and used freely in the optional arguments of `\item`, `\ptag`, `prop`, `inlineprop`, and the argument of `\propoptions`. Styles can freely be combined, and the definition of one style can reference another (in which case redefining the latter style will change the effect of the former style.)

`\SetPropStyle{`⟨*name*⟩`}{`⟨*keys*⟩`}`

Defines or modifies a prop style for use with the `style` key. All item-level keys (subsection 6.1) are accepted, plus the following:

`style=`⟨*parent*⟩ (no default)

Inherit from a parent style. When an item is created, the parent's settings are loaded first (recursively, if the parent itself has a parent), then this style's own keys are applied on top.

The argument may be a macro which is expanded when the item is created: for example, a style with `style=\{`⟨*style1*⟩`},{`⟨*style2*⟩`},{`⟨*style3*⟩`}` will resolve to `{`⟨*style1*⟩`}`, `{`⟨*style2*⟩`}`, `{`⟨*style2*⟩`}` depending on the nesting depth. The built-in `numbered` and `eqnum` styles use this mechanism.

`macro=`⟨*command*⟩ (no default)

A new user macro, equivalent to `\item[style=`⟨*name*⟩`]`. Any further keys given to the macro are passed to `\item`.

If the style ⟨*name*⟩ already exists, `\SetPropStyle` modifies or adds keys. For example, `\SetPropStyle{proposition}{align=flush}` changes the alignment of the built-in `proposition` style while preserving its other settings.

```
\SetPropStyle{angle}{
  labelindent    = 0em,
  display format = \textbf{$\langle$#1$\rangle$},
  ref format     = $\langle$#1$\rangle$,
  macro          = \angitem
}
\begin{prop}
```

```
  \angitem[Angle thesis] Everything is angular.
\end{prop}
No further discussion of \lastref{} is needed.
```

⟨**Angle thesis**⟩ Everything is angular.

No further discussion of ⟨Angle thesis⟩ is needed.

`\DeclareNumberedStyle{`⟨*name*⟩`}[`⟨*keys*⟩`]`

Creates a new LaTeX counter named ⟨*name*⟩ and a matching prop style with `counter=`⟨*name*⟩. All `\SetPropStyle` → P. 12 keys are accepted, plus:

`parent=`⟨*counter*⟩ (no default)

A parent counter; the new counter resets when the parent steps (same mechanism as `\numberwithin`). A dedicated `prop` counter (stepped by each `prop` and `inlineprop`) is available for non-persistent numbering.

```
\DeclareNumberedStyle{P}
\begin{prop}
  \item[counter=P] First premise. \label{p1}
  \item[counter=P] Second premise. \label{p2}
\end{prop}
From \ref{p1} and \ref{p2}\ldots
```

(P1) First premise.

(P2) Second premise.

From (P1) and (P2)...

`\proplevelchoice{`⟨*item1, item2, ...*⟩`}`

Picks the entry from the list depending on the current nesting depth (1-indexed, last element will be used if the list is too short). At depth 0 (outside any `prop` environment), returns the first entry. This macro is expandable, and designed to be used within `\SetPropStyle` to create styles that produce different results depending on the nesting depth of the `\item` in which they are used.

In a dimension key, an entry of `*` leaves that depth untouched (see section 6). A *blank* entry will not serve, and does not do what it looks like: the argument is read as a comma list, and comma lists discard empty items, so `\proplevelchoice{2em, , 1em}` has two entries rather than three and puts `1em` at depth 2.

Special counters `leveltwo`, `levelthree`, `levelfour`, `levelfive` are provided, whose values are automatically reset each time an `\item` is processed at a lower nesting level. These are especially useful in combination with `\proplevelchoice`, to generate hierarchically numbered styles: for examples, see the built-in `numbered`, `eqnum`, and `hierarchical` styles in the next section.

8 Built-in styles

The following prop styles are predefined. Each entry shows the defining code (using user-facing commands), and each group of styles ends with a live example. All styles can be modified with `\SetPropStyle` → P. 12; the definitions are reproduced here to facilitate modification.

Text styles

```
\begin{prop}
  \item[One, style=plain] \label{bs:plain}
  A proposition in style |plain|, cited as \ref{bs:plain}.
  \item[Two, style=proposition] \label{bs:prop}
  A proposition in style |proposition|, cited as \ref{bs:prop}.
  \titem[Three] \label{bs:thesis}
  A proposition in style |thesis|, cited as \ref{bs:thesis}.
  \item[Four, style=vignette] \label{bs:vignette}
  A proposition in style |vignette|, cited as \ref{bs:vignette}.
  \bitem \label{bs:bullet}
  A proposition in style |bullet|, cited as \ref{bs:bullet}---though
  it is not often that one would want to cite an item in this style.
\end{prop}
```

One A proposition in style `plain`, cited as One.

Two A proposition in style `proposition`, cited as **Two**.

THREE A proposition in style `thesis`, cited as Three.

Four: A proposition in style `vignette`, cited as *Four*.

- A proposition in style `bullet`, cited as (•)—though it is not often that one would want to cite an item in this style.

`plain` Unformatted text label.

```
\SetPropStyle{plain}{format = #1}
```

`proposition` Bold text label and reference. Auto-selected when `\item` has a name but no `style`.

```
\SetPropStyle{proposition}{format = \textbf{#1}}
```

`thesis` Small-caps name at the text margin (`flush` alignment). Shortcut: `\titem`.

```
\SetPropStyle{thesis}{display format = \textsc{#1},
  ref format=#1, align = flush, macro = \titem}
```

`vignette` Italic name at the text margin with a colon in the label.

```
\SetPropStyle{vignette}{ref format = \textit{#1},
  align = flush, label format = \textit{#1:}}
```

`bullet` Bullet symbol varying by depth (like `\itemize`). Shortcut: `\bitem`.

```
\SetPropStyle{bullet}{align = center,
  name = \proplevelchoice{\textbullet,
    {\normalfont\textendash}, \textasteriskcentered,
    \textperiodcentered},
  display format = #1, macro = \bitem}
```

Numbered styles

```
\begin{prop}
  \ritem \label{bs:roman}
  In style |roman|, cited as \ref{bs:roman}.
  \begin{prop}
    \ritem The same style in a sublist: the
    numbering restarts under each parent item.
    \ritem A second sub-item.
  \end{prop}
  \aitem \label{bs:alph}
  In style |alph|, cited as \ref{bs:alph}.
  \item[style=equation] \label{bs:equation}
  In style |equation|, cited as \ref{bs:equation}.
\end{prop}
```

(i) In style roman, cited as (i).
  (i) The same style in a sublist: the numbering restarts under each parent item.
  (ii) A second sub-item.

a. In style alph, cited as (a).

(8) In style equation, cited as (8).

`roman` Roman numerals. Shortcut: `\ritem`.

```
\DeclareNumberedStyle{roman}[parent = section,
  counter format = \roman{#1}, format = (#1), macro = \ritem]
\SetPropStyle{roman}{counter = \proplevelchoice{
  roman, numpropii, numpropiii, numpropiv, numpropv}}
```

`alph` Letters. Shortcut: `\aitem`.

```
\DeclareNumberedStyle{alph}[parent = section,
  counter format = \alph{#1}, display format=#1.,
  ref format = (#1), macro = \aitem]
\SetPropStyle{alph}{counter = \proplevelchoice{
  alph, numpropii, numpropiii, numpropiv, numpropv}}
```

`\DeclareNumberedStyle` → P. 13 gives a style the counter of the same name, which is what is wanted at level one: a sequence running through the document, reset at each `\section`. It is not what is wanted deeper down, where a sublist should start again under each parent item, so both styles override `counter` afterwards (a later key wins) to pick up the shared level counters instead. Either style can therefore number a sublist as well as a top-level list.

Two consequences worth knowing: below level one the numbering is shared with any other style using those counters, and the reference to a sub-item is just its own numeral, since the style's `format` applies at every level—unlike `leveltwo` and its siblings, whose `ref format` prefixes the parent.

`equation` Uses the `equation` counter. This style has a special behavior when the `equations` option is active: changing its `display format` and `ref format` keys (or both, by setting `format`) will also change the corresponding hooks used to create the tags for numbered equations make the `equation` environment and `amstex` environments like `gather`.

```
\SetPropStyle{equation}{counter = equation, format = (#1)}
```

The remaining numbered styles select one of the others according to the current nesting level.

```
\begingroup
\propoptions{style = hierarchical}
\begin{prop}
  \item \label{bs:h1} The world is everything that is the case.
  \begin{prop}
    \item \label{bs:h2} From \ref{bs:h1}, it follows that the
    underworld is everything that is not the case.
  \end{prop}
\end{prop}
\endgroup
\begin{prop}
  \item \label{bs:d1} Outside the group, this manual's own
  \texttt{eqnum} default for nameless items is back in effect.
  At this outermost level, it has the same effect as
  \texttt{equation}.  This item can be referenced as \ref{bs:d1}.
  \begin{prop}
    \item \label{bs:d2} Since this item (referenced as
    \ref{bs:d2}) is at level two, \texttt{eqnum} dispatches
    to \texttt{leveltwo}.
    \begin{prop}
      \item \label{bs:d3} Since this item (referenced as
      \ref{bs:d3}) is at level three, \texttt{eqnum}
      dispatches to \texttt{levelthree}.
    \end{prop}
  \end{prop}
\end{prop}
```

1. The world is everything that is the case.
   1.1 From 1, it follows that the underworld is everything that is not the case.

(9) Outside the group, this manual's own `eqnum` default for nameless items is back in effect. At this outermost level, it has the same effect as `equation`. This item can be referenced as (9).

a. Since this item (referenced as (9a)) is at level two, `eqnum` dispatches to `leveltwo`.

(i) Since this item (referenced as (9a.i)) is at level three, `eqnum` dis-

patches to `levelthree`.

These styles use `\proplevelchoice` to select a style depending on the current nesting level. Each is defined together with the level-specific styles it dispatches to. Those helper styles share the counters `numpropi` – `numpropv`, which reset automatically when an `\item` at the next lower level is processed.

`numbered` The default nameless style. Dispatches by nesting depth to `levelone` (arabic in parentheses), `leveltwo` (lowercase alphabetic with a dot), `levelthree` (lowercase roman in parentheses), `levelfour` (uppercase alphabetic with a dot) and `levelfive` (uppercase roman in parentheses).

```
\SetPropStyle{numbered}{style = \proplevelchoice{
  levelone, leveltwo, levelthree, levelfour, levelfive}}
\SetPropStyle{levelone}{counter = numpropi, format = (#1)}
\SetPropStyle{leveltwo}{counter = numpropii,
  display format = #1., ref format = \parentref{#1}}
\SetPropStyle{levelthree}{counter = numpropiii,
  display format = (#1), ref format = \parentref{.#1}}
\SetPropStyle{levelfour}{counter = numpropiv,
  display format = #1., ref format = \parentref{#1}}
\SetPropStyle{levelfive}{counter = numpropv,
  display format = (#1), ref format = \parentref{.#1}}
```

`eqnum` Like `numbered`, but uses `equation` at level 1. Activated by the `equations` package option (which sets `nameless style = eqnum`), and therefore the style in force for the unnamed items of this manual.

```
\SetPropStyle{eqnum}{style = \proplevelchoice{
  equation, leveltwo, levelthree, levelfour, levelfive}}
```

`hierarchical` Produces 1, 1.1, 1.1.1… numbering using `\parentref` and `counter format`. Defined via two helper styles:

```
\SetPropStyle{h-base}{counter = numpropi,
  counter format = \arabic{#1}, ref format = #1,
  display format = #1.}
\SetPropStyle{h-sub}{
  counter = \proplevelchoice{numpropi, numpropii,
    numpropiii, numpropiv, numpropv},
  counter format = \arabic{#1},
  format = \parentref{.#1}}
\SetPropStyle{hierarchical}{style = \proplevelchoice{
  h-base, h-sub}}
```

Note that since the optional arguments of the `prop` and `inlineprop` environment only affect the `\items` in the *immediate* scope of that environment (not of any nested environments), dispatcher styles are most useful when either set as defaults using keys like `nameless style` → P. 10, or set with `\propoptions` → P. 5 (whose scope can be controlled by creating a TEX group).

Environment-level styles

These styles are designed to be used in the optional argument of `prop` or `inlineprop`.

```
\begin{prop}[style=framed, style=thesis]
  \item[Central Doctrine] This thesis is so incredibly
  important that it deserves to be put in its own box!
\end{prop}
\begin{prop}[name=Fitted Margin, style=fitmargin]
  \item The name is set for the environment, and the
  body of the item begins where the label ends.
\end{prop}
```

CENTRAL DOCTRINE This thesis is so incredibly important that it deserves to be put in its own box!

Fitted Margin The name is set for the environment, and the body of the item begins where the label ends.

`framed` Frames a list in a `tcolorbox` (requires the tcolorbox package to be loaded). Change the padding with `\setlength\propframepad{...}`.

```
\SetPropStyle{framed}{
  align = flush, leftmargin = 0pt, rightmargin = 0pt,
  labelindent = {}, continue = false,
  wrapper begin = {\begin{tcolorbox}[
      colback = white, colframe = black,
      boxrule = 0.4pt, arc = 0pt, boxsep = 0pt,
      left skip  =
        \dimexpr\propoperativeleftmargin-\propframepad\relax,
      right skip =
        \dimexpr\propoperativeleftmargin-\propframepad\relax,
      left = \propframepad, right = \propframepad,
      top  = \propframepad, bottom = \propframepad]},
  wrapper end = {\end{tcolorbox}},
}
```

The `leftmargin=0` key means that the left margin *inside* the frame is zero. But the use of `\propoperativeleftmargin` → P. 11 indents the whole `tcolorbox` environment by the right amount for the the left margin of the enclosed material to appear at the same position that would have been used for a list without the `frame` style, with the frame rule one `\propframepad` further out.

`continue=false` keeps the frame clear of any list immediately before or after it. Joining does not merely retune the gap: it cancels the vertical glue already in place, which for a framed list is the tcolorbox's own spacing.

`fitmargin` Fits the left margin to the width of the label, so that each item's body begins where its label ends and the label sits in the hanging indent.

```
\SetPropStyle{fitmargin}{
  align      = left,
  labelwidth = \widthof{\propwidestlabel},
  leftmargin = \widthof{\propwidestlabel} + \labelsep}
```

This is what \propformlabel →P. 9 is for: it is the label this environment's items are expected to produce, computed once at \begin{prop} and therefore usable in the dimension keys that size the list. Both keys need it—leftmargin to place the body, and labelwidth to make the label box wide enough that the label starts at the enclosing margin rather than overflowing a box of the class default width. What they measure is \propwidestlabel →P. 9, the same label with its digits widened, so that a run of numbered environments does not shift its margin every time the numbers change width.

Since the name is set for the environment, every \item of the list carries the same label, so this style is meant for a list of one item; a numbered variant would want items →P. 10 as well, so that the width is computed for the last expected label rather than the first.

`outerfit` `fitmargin` at the outer level only, and nothing at all below it. Intended to be set once, for a document that wants all of its top-level propositions numbered in the manner of linguex:

```
\propoptions{style = outerfit}
```

Fitting the margin to the label is wanted at the outer level and unwanted inside it, where a nested list's roman numerals or letters would drag the margin about as they changed width. So each dimension takes a value that varies with the nesting level, using \proplevelchoice →P. 13:

```
\SetPropStyle{outerfit}{
  labelwidth = \proplevelchoice{
    \widthof{\propwidestlabel}, *},
  leftmargin = \proplevelchoice{
    \widthof{\propwidestlabel} + \labelsep, *}}
```

A dimension key's value is resolved where it is used, so \proplevelchoice picks the entry for the level actually being set up, and the * leaves every level below the first exactly as it would have been had the keys never been set: the document class's own dimensions, level by level. Nothing is frozen, and the same construction will scope any dimension to any range of levels.

Unlike fitmargin this style does not set align, which is left by default in any case: a style in force at every level should not overrule an alignment chosen further in.

`standard` Puts the list geometry back to the document class's own, for a single environment. Setting a style with \propoptions →P. 5 is easy; unsetting it is not, because a style applies the keys it names and leaves the rest alone—so asking for a *different* style does not restore what the first one did to the margins. Under \propoptions{style = outerfit}, for instance,

`\begin{prop}[name=Aside, style=proposition]` still gets the fitted margin. `standard` is the way back:

```
\begin{prop}[name=Aside, style=standard]
```

It sets every list dimension, every spacing key and `labelindent` → P. 10 to `*`, clears `wrapper begin` → P. 11 and `wrapper end` → P. 11, and restores `continue` → P. 10.

Geometry only: the label keeps whatever appearance its `named style` → P. 10 or `nameless style` → P. 10 gives it, so a named proposition reverted this way is still bold. That is a consequence of the order in which an item is resolved—the style chosen by `name` is loaded first, and the argument's keys are applied on top of it—so a style cannot reset a format without wiping the base style's formatting too. For the same reason item-level layout keys survive: to undo a style that also set `align` → P. 6, as `framed` does, add it to the argument: `[style=standard, align=left]`.

9 Cross-referencing commands

Labels placed after `\item` items (within `prop`) work with the standard `\label`/`\ref` mechanism. The key difference from ordinary LaTeX references is that `\ref` produces *formatted* output: for example, `\textbf` might be applied to the name, or the number might be wrapped in parentheses. The formatting is controlled by the `format` key (or separately by `display format` and `ref format`).

`\Ref{`⟨*label*⟩`}`
`\Ref*{`⟨*label*⟩`}`

Titlecase variants of `\ref` and `\ref*`: uppercases the first letter of the formatted output. (For example, if `\ref{thesis}` produces 'the Identity Theory', then `\Ref{thesis}` produces 'The Identity Theory'.) The starred form suppresses the hyperlink. Note: `\Ref` only affects references produced by this package (which are stored via `\propapply`); for other references, it behaves like `\ref`.

`\nref{`⟨*label*⟩`}`
`\nref*{`⟨*label*⟩`}`
`\Nref{`⟨*label*⟩`}`
`\Nref*{`⟨*label*⟩`}`

"Naked ref." Outputs the bare reference content with all formatting stripped. If `\ref{premise}` produces '(P1)', then `\nref{premise}` produces 'P1'. The starred form suppresses the hyperlink. `\Nref` is the titlecase variant: it uppercases the first letter of the bare content.

`\nref` can be useful in the argument of `\item`, when the the name of one proposition should depend on that of another:

```
\SetPropStyle{proposition}{format=(\textbf{#1})}
\begin{prop}
  \item[Phys] Everything is physical. \label{phys2}
  \item[\nref{phys2}*] Almost everything is physical.
```

```
  \label{newphys2}
  \item[\ref{phys2}*] This one has two sets of parentheses,
  which is probably not desired!  Note that
  the previous proposition \ref{newphys2} avoided this by using
  |\nref| in the optional argument
   of |\item|.
\end{prop}
```

(**Phys**) Everything is physical.

(**Phys***) Almost everything is physical.

((**Phys**)*) This one has two sets of parentheses, which is probably not desired! Note that the previous proposition (**Phys***) avoided this by using `\nref` in the optional argument of `\item`.

Warning: documents where the name of one item includes a reference to that of another, and there are further references to that item, will require multiple LaTeX runs to resolve all references. To save time, it is better to avoid long chains of dependencies of this sort.

`\oref[`⟨*prefix*⟩`][`⟨*suffix*⟩`]{`⟨*label*⟩`}`
`\oref*[`⟨*prefix*⟩`][`⟨*suffix*⟩`]{`⟨*label*⟩`}`
`\Oref[`⟨*prefix*⟩`][`⟨*suffix*⟩`]{`⟨*label*⟩`}`
`\Oref*[`⟨*prefix*⟩`][`⟨*suffix*⟩`]{`⟨*label*⟩`}`

"Ref with options." Extends `\ref` by injecting a prefix and/or suffix *inside* the formatting. With one optional argument, ⟨*suffix*⟩ is appended; with two, ⟨*prefix*⟩ is prepended and ⟨*suffix*⟩ appended. For instance, if `\ref{premise}` produces '(P1)', then `\oref[*]{premise}` produces '(P1*)' and `\oref[old~][*]{premise}` produces '(old~P1*)'.
`\Oref` is the titlecase variant; the starred forms suppress the hyperlink.

`\oref` can also be useful in the name of `\item`s, if one wants the display format for the modified item to depend on that originally used

```
\begin{prop}
  \label{premise}
  \item[style=plain, name=\oref[*]{phys2}]
  This will use boldface and parentheses because the
  original referenced item did.
\end{prop}
```

(**Phys***) This will use boldface and parentheses because the original referenced item did.

Another handy use for `\oref` is in combination with `\nref` to refer to ranges:

```
The first two numbered examples in this document
were \oref[--\nref{atomism}]{atoms}.
```

The first two numbered examples in this document were (1–2).

`\lastref[`⟨*prefix*⟩`]{`⟨*suffix*⟩`}`

`\Lastref[`⟨*prefix*⟩`]{`⟨*suffix*⟩`}`

Produces a reference (witth no hyperlink) to the most recently processed `\item` or `\ptag`, even without a `\label`. Useful for back-references in running text. With one argument, ⟨*suffix*⟩ is appended; with two, ⟨*prefix*⟩ is also prepended. Use `\lastref{}` for a plain reference. `\Lastref` is the titlecase variant.

`\nlastref`
`\nLastref`

Like `\lastref{}`, but returns the bare content without formatting. `\nLastref` is the titlecase variant.

`\parentref[`⟨*prefix*⟩`]{`⟨*suffix*⟩`}`
`\Parentref[`⟨*prefix*⟩`]{`⟨*suffix*⟩`}`

Inside a nested `prop` (or `inlineprop`), produces a formatted reference to the most recent item of the enclosing level. Same argument convention as `\lastref` → P. 21. `\Parentref` is the titlecase variant.

`\nparentref`
`\nParentref`

Like `\parentref{}` but returns the bare content without formatting. Takes no arguments; simply output any desired suffix directly afterwards. `\nParentref` is the titlecase variant.

`\parentref` and `\nparentref` are useful for making subitems whose names derive from their parent's:

```
\DeclareNumberedStyle{inner}[
  counter format=\alph{inner},
  display format=#1.]
\begin{prop}
  \item[OI] Outer item. \label{outer2}
  \begin{prop}
    \item[style=inner, format=\parentref{.#1}] \label{dsub1}
    Ref: \ref{dsub1}, naked: \nref{dsub1}.
    \item[style=inner,
          display format=#1., ref format=\parentref{.#1}]
    \label{dsub2}
    Ref: \ref{dsub2}, naked: \nref{dsub2}.
  \end{prop}
\end{prop}
```

OI Outer item.
OI.a Ref: **OI.a**, naked: a.
b. Ref: **OI.b**, naked: b.

The built-in `leveltwo` and `levelthree` styles have `ref format=\parentref{#1}` and `ref format=\parentref{.#1}`, respectively, so that if the parent references as '(**P1**)', a `leveltwo` sub-item references as '(**P1a**)' and `\nref` returns just 'a'.

9.1 How cross-referencing works: `\propapply`

`\propapply{`⟨*template*⟩`}{`⟨*content*⟩`}`

Internally, each reference is stored in the `.aux` file as `\propapply{`⟨*template*⟩`}{`⟨*content*⟩`}`. The ⟨*template*⟩ contains formatting with the placeholder `\propfmtarg` → P. 23 where content appears. At reference time, `\propapply` evaluates the template with `\propfmtarg` bound to ⟨*content*⟩. The `\oref` → P. 21 and `\nref` → P. 20 commands work by locally redefining `\propapply`.
In normal use, you need not interact with `\propapply` directly.

`\propfmtarg`

Placeholder used inside templates; expands to the content argument of the enclosing `\propapply` → P. 22.

10 Compatibility

The propositions package is designed to work with hyperref, cleveref, amsmath, and tcolorbox.

- amsmath is required for `\ptag` and the `equations` option.
- With hyperref loaded, all cross-referencing commands generate hyperlinks, as usual.
- With cleveref loaded, all proposition items are assigned to a private `prop` "reference type", which by default has no special name, so `\cref` and `\ref` will generate the same output. The `crefname` key can override this.
- The built-in `framed` style requires tcolorbox to be loaded.

None of these packages has to be loaded in any particular order relative to propositions: each is detected either at load time or at `\begin{document}`, whichever is needed. The usual external conventions still apply—hyperref late, and cleveref after hyperref—which gives:

```
\usepackage{amsmath}      % if using \ptag or the equations option
\usepackage{tcolorbox}    % if using the framed style
\usepackage{hyperref}
\usepackage{propositions}
\usepackage{cleveref}     % if used
```

11 Known issues

When using `\ptag` with a named counter (e.g. `\ptag[counter=P]`) inside an amsmath equation environment, hyperref may emit warnings of the form:

```
pdfTeX warning: destination with the same
identifier (name{equation.N}) has been already
used, duplicate ignored
```

These warnings are harmless and do not affect the correctness of cross-references.